

# 绿波走廊配置接口说明

## 1. 主要工作内容

绿波走廊配置函数部署与调用

- 通过 Python 函数查询、新增、更新、启用或停用绿波走廊。
- 走廊路口、阶段、时段、累计行程时间、路段映射和拓扑统一保存在 JSON 文件中。
- Global 每轮读取所有已启用走廊；配置保存成功后，下一轮处理自动生效。
- 本次交付为 Python 函数接口，不新增 HTTP 路径、监听端口或信控机发送逻辑。

## 2. 函数总览

| 函数                              | 作用           | 返回          |
|---------------------------------|--------------|-------------|
| list_green_wave_corridors       | 列出全部绿波走廊     | items 列表    |
| get_green_wave_corridor_config  | 查询一条走廊完整配置   | corridor 对象 |
| save_green_wave_corridor_config | 校验并新增或更新一条走廊 | 保存状态与规范化配置  |
| set_green_wave_corridor_enabled | 启用或停用一条走廊    | 启停结果        |

**说明：**接口位于 lib/green\_wave\_functions.py。配置文件为 lib/green\_wave\_corridors.json。

### 3. 通用调用约定

所有管理函数均接收一个 dict 类型的 payload，并返回 dict。

```
from lib.green_wave_functions import (
    list_green_wave_corridors,
    get_green_wave_corridor_config,
    save_green_wave_corridor_config,
    set_green_wave_corridor_enabled,
)
```

| 通用返回字段    | 类型   | 说明                                                    |
|-----------|------|-------------------------------------------------------|
| status    | str  | success、validated 或 error                             |
| saved     | bool | 本次调用是否已写入 JSON                                        |
| operation | str  | listed、queried、created、updated、<br>enabled 或 disabled |
| reason    | str  | 仅失败时返回错误原因                                            |

**说明：**函数返回错误字典，不要求调用方依赖异常捕获处理普通参数错误。

### 4. 查询接口

#### 4.1 list\_green\_wave\_corridors

列出 JSON 中全部走廊，包括已启用和已停用走廊。

| 参数      | 类型   | 必填 | 说明       |
|---------|------|----|----------|
| payload | dict | 是  | 传入空字典 {} |

```
result = list_green_wave_corridors({})
```

| 返回字段      | 类型   | 说明           |
|-----------|------|--------------|
| status    | str  | 成功时为 success |
| saved     | bool | 固定为 false    |
| operation | str  | 固定为 listed   |
| items     | list | 全部规范化走廊对象    |

## 4.2 get\_green\_wave\_corridor\_config

根据 corridor\_id 查询一条走廊完整配置。未传 corridor\_id 时默认查询 lvbo\_01。

| 参数字段        | 类型  | 必填 | 说明                |
|-------------|-----|----|-------------------|
| corridor_id | str | 否  | 走廊唯一编号，例如 lvbo_01 |

```
result = get_green_wave_corridor_config({  
    "corridor_id": "lvbo_01"  
})
```

| 返回字段      | 类型   | 说明           |
|-----------|------|--------------|
| status    | str  | 成功时为 success |
| saved     | bool | 固定为 false    |
| operation | str  | 固定为 queried  |
| corridor  | dict | 查询到的完整走廊配置   |

## 5. 新增或更新走廊

### 5.1 save\_green\_wave\_corridor\_config

该函数接收完整走廊对象。corridor\_id 不存在时新增，已存在时整体更新；不支持只传局部字段。

| payload 字段 | 类型   | 必填 | 说明                            |
|------------|------|----|-------------------------------|
| dry_run    | bool | 否  | true 仅校验；false 校验并保存，默认 false |
| corridor   | dict | 是  | 完整走廊配置对象                      |

建议先校验，再正式保存：

```
check_result = save_green_wave_corridor_config({
    "dry_run": True,
    "corridor": corridor_data,
})

save_result = save_green_wave_corridor_config({
    "dry_run": False,
    "corridor": corridor_data,
})
```

| 返回字段        | 类型   | 说明                                      |
|-------------|------|-----------------------------------------|
| status      | str  | dry_run=true 时为 validated；保存成功为 success |
| saved       | bool | 是否已写入 JSON                              |
| operation   | str  | created 或 updated                       |
| corridor_id | str  | 新增或更新的走廊编号                              |
| corridor    | dict | 校验并规范化后的完整配置                            |

**说明：**正式保存采用临时文件加 os.replace 原子替换；写入完成后会清理该走廊的进程内运行状态。

## 6. 启用或停用走廊

### 6.1 set\_green\_wave\_corridor\_enabled

| payload 字段  | 类型   | 必填 | 说明               |
|-------------|------|----|------------------|
| corridor_id | str  | 是  | 目标走廊编号           |
| enabled     | bool | 是  | true 启用；false 停用 |

```
result = set_green_wave_corridor_enabled({
    "corridor_id": "lvbo_02",
    "enabled": True,
})
```

**说明：** 停用不会删除走廊配置。两条已启用走廊不能包含同一个路口。

## 7. 走廊配置对象

### 7.1 基本字段

| 字段                              | 类型   | 必填 | 说明                        |
|---------------------------------|------|----|---------------------------|
| corridor_id                     | str  | 是  | 走廊唯一编号，只允许字母、数字、点、下划线和连字符 |
| name                            | str  | 否  | 走廊名称                      |
| enabled                         | bool | 是  | 是否参与 Global 协调            |
| cycle_seconds                   | int  | 是  | 统一完整周期，范围 30~600 秒        |
| offset_step_seconds             | int  | 否  | 单轮相位修正上限                  |
| minimum_remaining_green_seconds | int  | 否  | 车辆到达时要求保留的最小绿灯余量          |

| 字段               | 类型   | 必填 | 说明             |
|------------------|------|----|----------------|
| cooldown_seconds | int  | 否  | 两次调整之间的保护间隔    |
| intersections    | list | 是  | 至少两个路口         |
| periods          | list | 是  | 至少一个有效时间段      |
| road_junction    | dict | 建议 | 路段 RID 到路口方向映射 |
| topology         | dict | 建议 | 走廊路口相邻关系       |

### 7.2 intersections 路口字段

| 字段                  | 类型  | 说明                            |
|---------------------|-----|-------------------------------|
| cross_id            | str | 数字字符串形式的信控路口编号                |
| green_stage_index   | int | 绿波阶段在方案数组中的下标；阶段 1 对应 0       |
| balance_stage_index | int | 用于保持周期不变的补偿阶段下标，不能与绿波阶段相同     |
| default_red_seconds | int | 完整周期中的固定非绿灯时长，包括黄灯、全红或 -1 阶段等 |

```
{
  "cross_id": "1300999",
  "green_stage_index": 0,
  "balance_stage_index": 1,
  "default_red_seconds": 15
}
```

### 7.3 periods 时段字段

| 字段                    | 类型   | 说明                        |
|-----------------------|------|---------------------------|
| period_id             | str  | 时段唯一编号，例如 morning、evening |
| start_time / end_time | str  | HH:MM:SS；当前不支持跨午夜，时间段不能重叠 |
| reference_cross_id    | str  | 该方向参考路口，必须位于顺序第一位         |
| intersection_order    | list | 按车辆行驶顺序完整包含走廊全部路口         |
| travel_seconds        | dict | 从参考路口出发后的累计行程时间，参考路口必须为 0 |

**说明：**若三段路分别耗时 25、40、30 秒，四个路口应填写累计值 0、25、65、95，不能填写分段值 0、25、40、30。

### 7.4 road\_junction 路段映射

外层键为路段数据中的 rid，内层键只允许 L、R、U、D，值必须是本走廊路口编号。

```
"road_junction": {
  "RID_A": {"R": "1300999"},
  "RID_B": {"L": "1300999", "R": "1301000"}
}
```



**说明：**填写 road\_junction 后，走廊内每个路口至少要被一条 RID 映射到；同一连接路段可以同时关联两端路口。

## 7.5 topology 路口拓扑

走廊内每个路口都填写 L、R、U、D 四个方向的相邻路口；没有相邻路口时填空字符串。

```
"topology": {  
  "1300999": {"L": "", "R": "1301000", "U": "", "D": ""},  
  "1301000": {"L": "1300999", "R": "", "U": "", "D": ""}  
}
```

## 8. 新增一条绿波走廊

1. 准备走廊编号、完整周期、控制路口和各路口阶段下标。
2. 按每个启用时段填写行驶顺序和累计行程时间。
3. 填写相关路段 RID、路口方向映射和完整拓扑。
4. 先以 enabled=false、dry\_run=true 调用保存接口进行校验。
5. 校验通过后 dry\_run=false 正式写入，再调用启停接口启用。

完整示例：

```

corridor_data = {
  "corridor_id": "lvbo_02",

  "name": "新绿波走廊",

  "enabled": False,
  "cycle_seconds": 90,
  "offset_step_seconds": 3,
  "minimum_remaining_green_seconds": 10,
  "cooldown_seconds": 85,
  "intersections": [
    {"cross_id": "1300999", "green_stage_index": 0,
     "balance_stage_index": 1, "default_red_seconds": 15},
    {"cross_id": "1301000", "green_stage_index": 0,
     "balance_stage_index": 1, "default_red_seconds": 15}
  ],
  "periods": [{
    "period_id": "morning",
    "start_time": "07:30:00", "end_time": "09:00:00",
    "reference_cross_id": "1300999",
    "intersection_order": ["1300999", "1301000"],
    "travel_seconds": {"1300999": 0, "1301000": 35}
  }],
  "road_junction": {
    "RID_A": {"R": "1300999"},
    "RID_B": {"L": "1300999", "R": "1301000"}
  },
  "topology": {
    "1300999": {"L": "", "R": "1301000", "U": "", "D": ""},
    "1301000": {"L": "1300999", "R": "", "U": "", "D": ""}
  }
}

```

## 9. 向现有走廊增加一个路口

不能只在 intersections 中追加路口编号。必须取得原完整配置，完成以下五项后整体保存：

1. 在 intersections 中增加路口阶段和固定非绿灯时长。

- 2. 在每个 periods.intersection\_order 中插入正确位置。
- 3. 在每个 travel\_seconds 中增加该路口的累计行程时间。
- 4. 在 road\_junction 中增加与该路口有关的 RID 和方向。
- 5. 在 topology 中增加该路口，并修正前后相邻关系。

**说明：**保存接口执行完整对象更新，不是 PATCH。少改任何一项都可能导致校验失败或拥堵数据无法关联。

10. 当前 lvbo\_01 配置摘要

| 项目      | 当前值                                   |
|---------|---------------------------------------|
| 晚高峰顺序   | 2705050 → 1300370 → 1300373 → 1300248 |
| 晚高峰累计时间 | 0、26、65、98 秒                          |
| 早高峰顺序   | 1300248 → 1300373 → 1300370 → 2705050 |
| 早高峰累计时间 | 0、33、72、98 秒                          |
| 完整周期    | 90 秒                                  |
| 路段映射    | 5 条 RID，配置在 road_junction             |

11. Global 生产调用链

```
green_wave_corridors = load_enabled_green_wave_corridors()

for corridor in green_wave_corridors:
    result_action_map = lvbotest.apply_green_wave_coordination(
        time_str=time_str,
        cur_action_map=cur_action_map,
        result_action_map=result_action_map,
        coordinate_map_set=coordinate_map_set,
        extend_map=extend_map,
        corridor_config=corridor,
    )

action, result_check_report = phase_check(result_action_map)
record_green_wave_final_plan(action)
```

**说明：**新走廊保存并启用后，Global 下一轮自动加载。若 result\_action\_map 缺少该走廊任一  
路口方案，本轮跳过并输出缺失路口。

12. 配置校验与错误处理

| 校验场景                | 处理结果                  |
|---------------------|-----------------------|
| 路口少于两个或编号不是数字字符串    | 返回 status=error，不写入文件 |
| 绿波阶段与补偿阶段相同         | 拒绝保存                  |
| 时间段重叠、跨午夜或顺序未覆盖全部路口 | 拒绝保存                  |
| 累计行程时间未从 0 开始或未严格递增 | 拒绝保存                  |

| 校验场景                       | 处理结果                   |
|----------------------------|------------------------|
| RID 方向不是 L/R/U/D，或映射到走廊外路口 | 拒绝保存                   |
| topology 缺少走廊路口或包含无效方向     | 拒绝保存                   |
| 两条启用走廊共用同一路口               | 拒绝启用或保存                |
| corridor_id 不存在            | 查询或启停接口返回 status=error |

### 13. 部署与并发要求

- 接口读写项目目录下同一份 lib/green\_wave\_corridors.json。
- 同一 Python 进程内使用可重入锁保护读取和保存；文件采用原子替换。
- 建议由一个配置管理入口执行写操作，避免多个进程同时覆盖同一文件。
- 新路口必须已经进入原业务处理链，并能在 result\_action\_map 中生成阶段方案。
- JSON 只负责绿波配置，不替代路口基础接入、方案生成、phase\_check 或原发送流程。

### 14. 交付边界

- 提供绿波 Python 配置函数及其在 Global 中的调用方式。
- 不新增 HTTP 路径、不新增监听端口、不改变现有 TCP 或雷达 HTTP 服务。
- 配置函数不直接向信控机发送数据，也不绕过 phase\_check。
- 最终方案仍由现有 Server\_AITC.py 处理链生成并下发。